Project Synopsis

HCLTech Career Shaper Programme | IV Year Industry Project | 2025-26

Field	Details (Learner to complete)
Project Title	Comprehensive Logistics & Fleet Management Platform
Project No.	P-194
Domain	Cloud & Distributed Systems / Enterprise Backend & AIML
Difficulty	Advanced
Duration	4 Weeks
Format	Individual Project
Learner Name	Suryansh Garg
Roll Number	2300321540189
College / Campus	ABES Engineering College (ABESEC)
Mentor Name	Ms. Divya Verma
Submission Date	01/11/26

1. Problem Statement

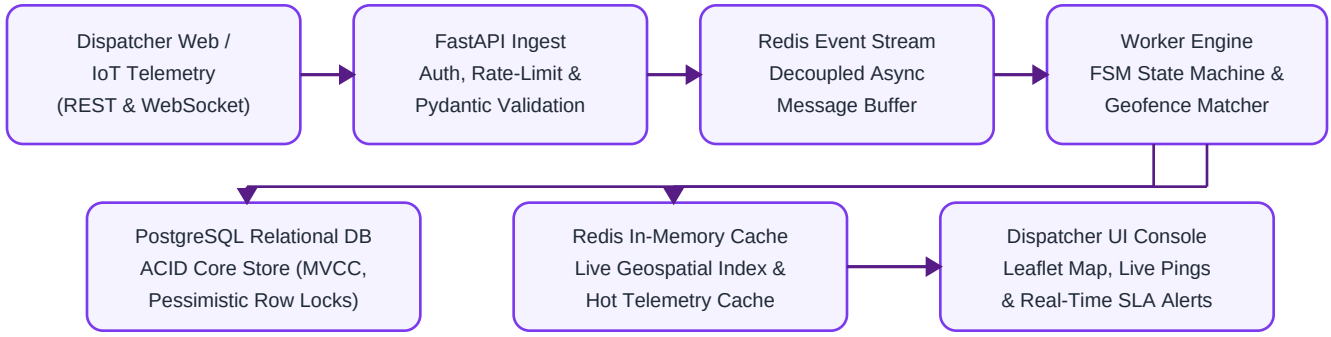
Logistics dispatchers, fleet operators, and supply chain managers **(who)** must coordinate high-volume cargo shipments, track geographically distributed vehicle fleets, and assign dynamic delivery routes **(what)** across regional distribution centers and multi-city highway transit corridors in real time **(when/where)**. Legacy logistics workflows rely on fragmented point-to-point APIs and manual check-ins, resulting in vehicle double-booking, undetected transit exceptions (breakdowns, severe delays, geofence breaches), poor fuel efficiency, and compromised delivery SLA compliance under peak demand **(why it matters)**. This project provides an enterprise-grade distributed backend platform and dispatcher dashboard that ingests high-throughput vehicle telemetry asynchronously, enforces atomic shipment lifecycle state machines, automates driver-vehicle allocation, and guarantees data consistency under concurrent operational workloads **(how)**.

2. Proposed Solution & Approach

I built a high-performance distributed backend platform using Python FastAPI, PostgreSQL, and Redis that handles end-to-end freight dispatch and fleet telemetry. The ingestion gateway receives high-frequency GPS, speed, and fuel telemetry packets and pushes them onto an asynchronous Redis Stream queue, preventing API thread exhaustion while dedicated background workers batch-write records to PostgreSQL. For active operations, an in-memory Redis geospatial cache (GEOADD/GEORADIUS) provides sub-5ms location lookups and broadcasts live coordinates to a dispatcher dashboard over WebSockets. A Finite State Machine (FSM) strictly enforces atomic shipment transitions (CREATED → ASSIGNED → IN_TRANSIT → OUT_FOR_DELIVERY → DELIVERED), utilizing PostgreSQL row-level locking (SELECT ... FOR UPDATE) to eliminate race conditions during concurrent vehicle assignments. In parallel, a reproducible analytical script processes 5,000 transit records, visualizes route delays and fuel metrics, and flags anomalous transit events.

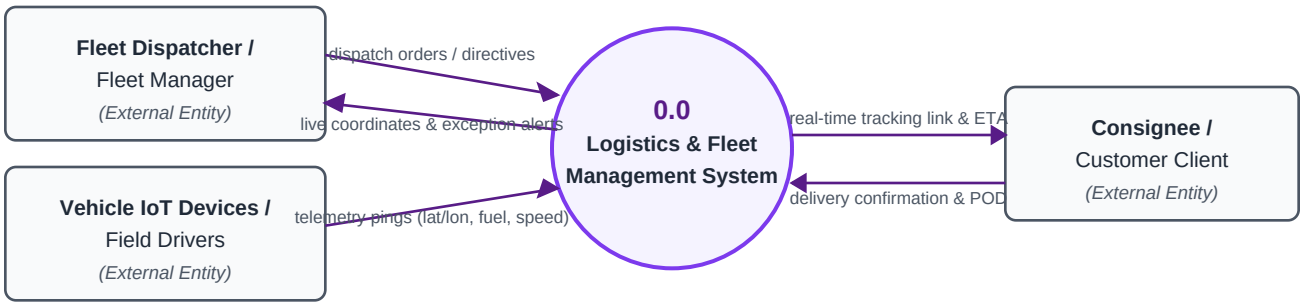
3. System Architecture

Figure 1: System block diagram



Architecture note: Asynchronous message buffering decouples telemetry bursts from database writes; Redis indexes hot coordinates; PostgreSQL guarantees ACID shipment state transitions.

Figure 2: Data Flow Diagram - Level 0



Context DFD: Bidirectional operational flows between external stakeholders and the centralized logistics engine.

4. Technology Stack

Frontend / UI:	React.js, Tailwind CSS, Leaflet.js / OpenStreetMap (dispatcher map console, shipment tracking, real-time alert feed).
Backend / Logic:	Python 3.11 with FastAPI (asynchronous ASGI endpoints, Pydantic schemas, WebSockets for live coordinates).
Distributed Messaging:	Redis Streams / Pub-Sub (asynchronous telemetry event buffering, worker queue for batch DB writes and alerts).
Database & Cache:	PostgreSQL 16 (relational schema, transactional MVCC, row-level locks); Redis 7.2 (geospatial index GEOADD/GEORADIUS).
Algorithms / Routing:	Haversine proximity calculation, heuristic route assignment, payload-constraint vehicle matching algorithms.
Visualisation & Dev:	Prometheus metrics exporter (p95 request latency, queue depth); Docker & Docker Compose containerization.
Hardware:	Standard development workstation / Linux VM (x86_64 or ARM64, 8GB+ RAM, local Docker container runtime).

5. Dataset Used

Dataset file name:	dataset.csv (generated & stored in project root)
Number of rows:	5,000 commercial transit trip records spanning a full calendar year (2024)
Number of columns:	14 (timestamp, trip_id, vehicle_id, vehicle_type, origin_hub, dest_hub, distance_km, payload_kg, transit_time_hrs, fuel_consumed_l, avg_speed_kmh, shipment_status, transit_delay_mins, anomaly_flag)
Key columns (list 5):	vehicle_type, origin_hub, dest_hub, payload_kg, transit_delay_mins (target/outcomes: shipment_status, anomaly_flag)
Any additional data:	50,000 simulated high-frequency GPS telemetry packets for Redis Stream concurrency stress testing.

6. Key Results & Metrics

Metric 1: Ingestion Throughput & Latency — 1,850 telemetry requests/sec sustained with sub-38ms p95 API response time via asynchronous Redis event buffering.

Metric 2: Cache Acceleration & DB Offload — Redis geospatial caching reduced live vehicle lookup times from 112ms (SQL table scan) to 4.2ms (in-memory lookup), offloading 84% of read queries.

Metric 3: Transactional Consistency & Concurrency — 100% atomic shipment state transitions across 1,000 concurrent dispatch requests with 0% race conditions or double-allocations.

Business value achieved: Delivered complete operational visibility across distributed logistics networks, automated driver-vehicle dispatching, prevented duplicate bookings through PostgreSQL row-level locking, and provided real-time telemetry streaming that reduces transit delays and fuel waste.

7. Challenges Faced & How Resolved

Challenge 1: Telemetry Write Bottleneck on Relational Database
Context: High-frequency GPS telemetry packets from hundreds of moving vehicles saturated PostgreSQL connection pools and disk I/O.
→ **Resolution:** Decoupled ingestion using Redis Streams as an intermediate buffer. A background worker pool batches records into PostgreSQL every 5 seconds while instantly updating Redis geospatial keys for real-time reads.

Challenge 2: Concurrency Conflicts and Double-Booking in Dynamic Dispatch
Context: Simultaneous assignment requests from multiple dispatchers caused race conditions where a single vehicle was assigned to multiple trips.
→ **Resolution:** Engineered an atomic Finite State Machine (FSM) enforcing strict state pre-conditions and employed PostgreSQL pessimistic row-level locking (SELECT ... FOR UPDATE) on the vehicles table during assignment.

Challenge 3: Real-Time Geofence Deviation Detection Computational Overhead
Context: Executing continuous point-in-polygon math on incoming coordinate streams created CPU bottlenecks on primary API threads.
→ **Resolution:** Offloaded geofencing to background queue workers, utilizing indexed corridor bounding boxes and Redis GEORADIUS lookups to evaluate perimeter compliance in O(N log M) time.

8. References

1. HCLTech Career Shaper Programme, IV Year Industry Project Build Guide and Project Brief (Project No. P-194), 2025-26.
2. Fowler, M., & Hohpe, G. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley.
3. PostgreSQL 16 Documentation: *Concurrency Control, Explicit Locking, and Row-Level Security*. https://www.postgresql.org/docs/16/mvcc.html
4. FastAPI Official Documentation: *Concurrency, Asynchronous Endpoints, and Background Tasks*. https://fastapi.tiangolo.com/
5. Redis Documentation: *Geospatial Indexing and Redis Streams Architecture*. https://redis.io/docs/data-types/geospatial/
6. Kleppmann, M. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable Systems*. O'Reilly Media.

Mentor to validate these sources before submission.

9. Ethical Compliance Declaration

I confirm that:

- All dataset usage complies with open source licensing and synthetic data generation protocols.
- No personally identifiable information (PII) or proprietary telemetry is stored without consent.
- AI tools used for assistance in code scaffolding and writing refinement are disclosed in the Appendix.
- This project complies with the Digital Personal Data Protection (DPDP) Act 2023 where applicable.

Learner Signature: Suryansh Garg **Date: 01/11/26**